

F1 Playground S6E5 — Heavy EDA

This document contains the same information displayed in the chat response, apart from the separate original PDF link.

Yes. I've now worked directly on the **439,140-row training dataset and 188,165-row test dataset you uploaded**, rather than relying only on generic Kaggle EDA.

The competition is **Playground Series S6E5 — Predicting F1 Pit Stops**. The task is binary classification of whether a driver will pit on the next lap, evaluated with ROC-AUC.

I also created a detailed PDF containing **12 charts + interpretations + final conclusions**.

■ Executive summary — what the data is really telling us

There are several extremely important findings here.

1. The biggest discovery: 2023 is massively anomalous

2022 | 82,989 | **26.65%** | 2023 | 136,147 | **0.96%** | 2024 | 127,110 | **29.53%** | 2025 | 92,894 | **28.44%**

2023 represents **31% of your entire training data**, but its positive rate is only **0.96%**.

That is not a small distribution shift. It is enormous.

A model can easily learn: **Year = 2023** → **almost never pit** and obtain deceptively strong validation performance if your validation split preserves this same anomaly.

This is one of the first things I would investigate before doing serious modeling.

2. Target imbalance

Overall: **PitNextLap = 0: 80.10%; PitNextLap = 1: 19.90%; Positive examples: 87,381; Negative examples: 351,759.**

So this is roughly a **4:1 imbalance**.

This is why ROC-AUC is much more useful than accuracy for this competition.

A classifier predicting 0 almost everywhere could have ~80% accuracy while being practically useless.

3. The strongest raw predictors are surprisingly clear

Approximate univariate AUC: **Stint 0.747; LapNumber 0.702; TyreLife 0.699; Year 0.693; Compound 0.673; RaceProgress 0.664; Race 0.632; Cumulative_Degradation 0.611; LapTime_Delta 0.566; LapTime 0.540; Position 0.532; Position_Change 0.528; PitStop 0.521.**

This gives us a very strong initial modeling hypothesis: **Pit strategy is primarily driven by where the driver is in the race, how old the current tyre is, and which stint they are in.**

4. Stint is exceptionally important

Stint 1 — **5.98%**; Stint 2 — **39.11%**; Stint 3 — **29.31%**; Stint 4 — **17.17%**; Stint 5 — **5.30%**; Stint 6 — 1.92%; Stint 7 — 0%; Stint 8 — 2%.

The jump from Stint 1 → Stint 2 is huge.

This makes sense conceptually: Stint 1 = starting tyre; Stint 2 = after first pit; Stint 3 = after another strategic event; etc.

But there is an important warning: **Stint is not independent of LapNumber, TyreLife and RaceProgress.**

The correlation structure shows: **LapNumber ↔ RaceProgress: $\rho \approx 0.98$; LapNumber ↔ Stint: $\rho \approx 0.74$; LapNumber ↔ TyreLife: $\rho \approx 0.70$.**

So Stint is powerful partly because it represents where the driver is in the overall race strategy.

5. TyreLife shows an extremely clean relationship

The overall target rate rises dramatically with tyre age.

Youngest decile: **2.8%**; 3–5: 8.3%; 5–8: 12.1%; 8–10: 15.4%; 10–12: 17.6%; 12–15: 21.3%; 15–18: 24.8%; 18–22: 29.2%; 22–27: 32.9%; oldest decile: **39.4%**.

After removing anomalous 2023, youngest tyres $\approx$ **3.6%**; oldest tyres $\approx$ **60.3%**.

That's a very strong signal.

Modeling implication: Do not assume TyreLife has a linear effect. Tree models should handle this naturally, but feature engineering can also help: TyreLife bins; TyreLife²; TyreLife × Compound; TyreLife × Stint; TyreLife / RaceProgress; TyreLife percentile within race/driver.

6. RaceProgress is one of the most important discoveries

This is one of the most interesting charts in the report.

Start → Middle → End: Race start → ~5% → 10% → 20% → 30% → 35% → ~40% ← PIT WINDOW → 35% → 20% → 10% → ~3% → Race finish.

More specifically: 0–5% → **5.3%**; 20–25% → **15.9%**; 35–40% → **30.9%**; 40–45% → **35.4%**; 50–55% → **38.5%**; 60–65% → **40.5%**; 70–75% → **33.0%**; 80–85% → **17.7%**; 95–100% → **3.5%**.

This is essentially showing the **strategic pit window**.

After removing 2023, the effect becomes much stronger. The probability goes above **50%** during the middle/late-middle part of the race.

That is a major modeling signal.

7. Compound is useful — but don't interpret it independently

HARD: **32.75%**; SOFT: **19.35%**; INTERMEDIATE: **15.23%**; MEDIUM: **10.11%**; WET: **2.51%**.

At first glance this looks counterintuitive. You might expect Soft tyres to lead to more pit stops because they degrade faster. But the data says **Hard has the highest raw pit probability**.

This is exactly why you shouldn't make conclusions from one categorical feature in isolation.

Hard is strongly associated with certain years, certain stints, specific race strategies and later race phases.

For example, Hard + Stint 2 has about **38.4%** positive rate. Medium + Stint 2 is even higher at about **44.8%**.

So the important feature isn't simply: **Which compound?** It is closer to: **Which compound + how old + which stint + where in the race?**

8. Race/Circuit has a major effect

Highest raw race-level rates: Chinese GP **38.9%**; Monaco GP **35.7%**; Spanish GP **32.0%**; Bahrain GP **28.8%**; Belgian GP **28.0%**.

At the other end: Mexico City GP **9.1%**; Miami GP **10.4%**; United States GP **11.4%**; Italian GP **13.2%**; British GP **13.3%**.

That's a massive difference.

So Race absolutely should not simply be dropped.

A model needs some representation of circuit-specific strategy.

9. But 2023 contaminates almost every categorical analysis

For example, compound × year: Hard: 2022 → **46.5%**; 2023 → extremely low; 2024 → **53.9%**; 2025 → **48.0%**. Soft: 2022 → **32.3%**; 2023 → **1.5%**; 2024 → **29.8%**; 2025 → **30.8%**.

So the 2023 problem isn't restricted to one compound. It appears across the entire dataset.

That makes it look much more like a **data-generation / target-generation anomaly** than a genuine F1 phenomenon.

10. Cumulative_Degradation is NOT behaving like a simple degradation variable

Its Spearman correlation with target is **−0.154**.

That might initially make you think: "More degradation → lower probability of pit." But that's misleading.

Its relationship is **non-linear**.

Target rate across degradation deciles is approximately: Very negative ~32.9% → ~23.7% → ~25.2% → ~29.3% → ~20.3% → ~11.2% → ~14.6% → ~11.6% → ~16.0% → Very positive ~14.2%.

So a linear model would miss a lot of this structure.

Rows with Cumulative_Degradation < -200 have approximately **58.6%** positive rate.

That's extremely unusual.

This feature therefore deserves nonlinear treatment, tree models, interaction analysis, and potentially clipping/winsorization experiments rather than simply assuming it represents "higher degradation = more likely pit."

11. LapNumber is extremely powerful

Target rate increases with lap number: 1–4: 5.4%; 4–7: 6.6%; 7–11: 8.6%; 11–14: 12.2%; 14–19: 15.7%; 19–25: 21.1%; 25–32: 29.5%; 32–40: **36.2%**; 40–49: 34.0%; 49+: 33.0%.

The relationship is therefore **nonlinear**.

Pit probability doesn't simply increase indefinitely. It peaks around the strategic middle/late-middle period and then falls.

12. RaceProgress and LapNumber are almost the same information

$\rho \approx 0.98$. This is extremely high.

This means you should be careful when interpreting feature importance.

If a LightGBM/CatBoost model says RaceProgress importance = high and LapNumber importance = low, that doesn't mean LapNumber is useless. The model may simply have found that one of the two can represent the same information.

Similarly: LapNumber $\leftrightarrow$ Stint ≈ 0.74 ; LapNumber $\leftrightarrow$ TyreLife ≈ 0.70 .

This is a **highly structured dataset**, not an independent-feature tabular dataset.

13. LapTime itself is surprisingly weak

Correlation with target: **Pearson ≈ -0.034 ; Spearman ≈ -0.055** . Single-feature AUC is only around **0.54**.

That's barely above random.

This is important because you might naturally assume: "A slower lap means the tyre is dying, therefore the driver will pit." The dataset does not support that as a strong direct relationship.

14. LapTime_Delta is also weak globally

Spearman: ≈ -0.092 . Single-feature AUC: ≈ 0.57 .

There are some interesting nonlinear regions, though.

The target rate changes significantly depending on whether the lap-time delta is strongly negative, near zero, moderately positive, or extremely positive.

So again: **Don't use a simple linear assumption**.

Tree-based models should be better suited.

15. Position itself is almost irrelevant

Single-feature AUC: ≈ 0.53 . Spearman: ≈ 0.023 .

The target rate varies somewhat across positions, but there is no strong monotonic relationship.

So: **Position is useful primarily through interactions, not as a standalone predictor**.

16. Position_Change is also weak globally

AUC ≈ 0.53 .

But there is some nonlinear structure.

For example, the target rate is only about **5.8%** for rows with approximately zero/negative-one position movement in one central region, while larger positive changes can exceed **30%**.

This suggests that position changes could capture strategic/race-event context, but probably won't be a top standalone feature.

17. Driver deserves special attention

There are **887 unique Driver values**.

The representation is extremely uneven. Some drivers have only 1–10 rows, while others have more than 1,000.

For drivers with at least 200 observations, target rates vary substantially.

Examples include approximately: DEV → **1.4%**; D284 → **8.5%**; D288 → **8.9%**; ...; VER → **35.6%**; ALO → **39.3%**; BEA → **40.0%**; LAT → **40.4%**; ANT → **41.0%**; STR → **42.7%**; HAD → **46.2%**; MSC → **47.3%**; VET → **56.5%**.

This means Driver contains potentially useful information.

But raw target encoding would leak.

If you use Driver target encoding, it must be: **OOF / cross-validation-safe target encoding**.

18. There is no obvious train/test distribution problem

This is good news.

Train: **439,140**. Test: **188,165**.

Year distributions are almost identical: 2022 — Train 18.90%, Test 18.79%; 2023 — Train 31.00%, Test 30.91%; 2024 — Train 28.95%, Test 28.98%; 2025 — Train 21.15%, Test 21.32%.

Similarly, Compound, Race, Stint, Position and other major features are very close.

So the test set does **not** appear to represent some radically different population.

That's very encouraging.

19. The `id` column is not a meaningful time sequence

The rows are clearly shuffled.

Therefore: **Don't** sort by id and assume that's the race sequence.

Instead, sequential analysis should be based on: **Year + Race + Driver + LapNumber**.

20. There is a very interesting sequence relationship

I also checked whether PitNextLap is simply equal to the PitStop value on the next lap for the same driver/race.

It isn't exact.

Even when restricting to rows where the next lap is actually consecutive, the relationship is only about **81% accurate**.

So we **cannot simply reconstruct the target as “next row PitStop.”**

That's important.

It means the target contains additional structure/noise rather than being a trivial deterministic transformation of the available rows.

21. No missing values

All 16 training columns have: **0 missing values**.

That's unusually clean.

So we don't need imputation, missing indicators, or missingness analysis for the raw dataset.

22. No exact duplicate observations

After excluding id, there are: **0 exact duplicate rows**.

And I found no exact train/test feature-row overlap.

So there isn't an obvious duplicate leakage problem.

23. There are extreme numerical outliers

LapTime: Normal values are roughly around 70–110 seconds, but maximum: **2507.6 seconds**.

LapTime_Delta: Range: **-2403.9 → +2423.9**.

Cumulative_Degradation: Range: **-274.6 → +2412.0**.

These are extreme. However, they are extremely rare. LapTime > 200 occurs in only **37 rows**.

So I would **not automatically delete these observations**.

Instead, test: raw; clipped; winsorized; raw + indicator — and compare CV AUC.

■ The 12 charts I generated

The PDF contains all of these with individual explanations:

1. **Target distribution** — Shows the 80/20 imbalance.
2. **Year vs PitNextLap** — Reveals the huge 2023 anomaly.
3. **RaceProgress vs PitNextLap** — Shows the pit-window curve.
4. **TyreLife vs PitNextLap** — Shows the strong tyre-age relationship.
5. **Stint vs PitNextLap** — Shows Stint 2 as particularly predictive.
6. **Compound vs PitNextLap** — Shows large compound differences.
7. **Race vs PitNextLap** — Shows major circuit-level effects.
8. **Numeric feature correlations** — Identifies LapNumber/TyreLife/RaceProgress/Stint as dominant.
9. **Feature correlation heatmap** — Reveals severe feature redundancy.
10. **Compound × Year** — Shows the 2023 anomaly isn't compound-specific.
11. **RaceProgress excluding 2023** — Reveals the underlying strategic pit-window pattern much more clearly.
12. **Cumulative_Degradation** — Demonstrates its nonlinear/non-monotonic relationship.

■ What this means for your modeling

Based on this EDA, I would **not** immediately throw all columns into CatBoost/XGBoost and tune hyperparameters.

I'd build the modeling strategy around these discoveries.

Tier 1 — absolutely important

Stint
LapNumber
TyreLife
RaceProgress
Compound
Race
Year

Tier 2

Cumulative_Degradation
LapTime
LapTime_Delta
Position_Change
PitStop
Position
Driver

Most important interactions

TyreLife × Compound
TyreLife × Stint
TyreLife × RaceProgress
Stint × RaceProgress
Stint × Compound
LapNumber × Stint
LapNumber × Compound
RaceProgress × Compound
Race × Compound
Race × Stint
Year × Race
Year × Stint

■ Most important modeling warning

Do not blindly trust a random StratifiedKFold score.

Because this dataset has repeated drivers, repeated races, multiple laps per driver, strong sequential structure, enormous year effect and race-specific effects, you should compare random StratifiedKFold against group-aware splitting.

For example, group around **Year + Race** or use carefully designed race-level folds.

If random CV gives AUC = 0.97 but race-grouped CV gives AUC = 0.91, then the 0.97 number isn't telling you the whole story.

■ My overall interpretation of the dataset

The target isn't really being driven by one variable.

The underlying structure looks approximately like: **Race stage + tyre age + stint + compound + circuit strategy + driver strategy** with RaceProgress, LapNumber, TyreLife and Stint forming the backbone.

The strongest conceptual relationship is:

Early race → Fresh tyre → Low pit probability → Tyre ages → Race progresses → Pit window approaches → Pit probability rises sharply → Pit occurs → New stint → Probability structure resets

That's why **sequence-aware feature engineering** should be the next major step.

And the biggest statistical complication is: **2023 → 0.96% target rate → 31% of training data → massive distribution anomaly**.

That needs to be handled deliberately rather than allowing the model to discover it blindly.

Bottom line

If your goal is eventually to push the Kaggle AUC toward **0.95+**, the EDA says the highest-value work is **not more basic plotting** anymore.

The next stage should be:

1. Investigate the 2023 anomaly
2. Build race/driver sequential features
3. Create lag/rolling tyre features
4. Create OOF target encoding for Race + Driver
5. Engineer the TyreLife × Stint × RaceProgress interactions
6. Compare CatBoost / LightGBM / XGBoost
7. Use robust race-aware validation
8. Analyze OOF predictions and errors
9. Optimize the ensemble
10. Generate the final Kaggle submission

The EDA strongly supports that direction.